

ANALYSIS AND SIMULATION OF A QPSK SYSTEM

EQ2310 Digital Communications

Student 1: Oguzhan Oguz

Person number: 20020308-6293

☒ Student 1 has worked alone, there is no Student 2.

(Fields below filled-in by the teacher)

	PASSED	FAILED
length of report (number of pages)	☐ ≤ 6 pages ☐ > 6	☐ ≤ 6 pages ☐ > 6
number of authors	☐ ≤ 2 ☐ > 2	☐ ≤ 2 ☐ > 2
author name(s) and personal id number(s) stated	☐ yes ☐ no	☐ yes ☐ no
the text is easy to follow and the language correct	☐ yes ☐ mostly ☐ most of them	☐ yes ☐ only a few ☐ no
results and conclusions are clearly stated	☐ yes ☐ mostly ☐ most of them	☐ yes ☐ only a few ☐ no
results are correct	☐ yes ☐ mostly ☐ not	☐ yes ☐ only a few ☐ no
the extent of the work is satisfactory	☐ yes ☐ mostly ☐ not	☐ yes ☐ only a few ☐ no
the results can be reproduced	☐ yes ☐ mostly ☐ no	☐ yes ☐ hard to tell ☐ no
the mathematics/theory is correct	☐ yes ☐ mostly ☐ no	☐ yes ☐ hard to tell ☐ no
first check	☐ P	☐ F Sign:
second check (when F above)	☐ P	☐ F Sign:
PASSED	Signature:	

Abstract

This report presents a MATLAB-based analysis and simulation of a Gray-coded QPSK baseband communication link over an AWGN channel. The implemented chain includes pulse shaping, matched filtering, training-based timing synchronization, training-based phase estimation, and hard-decision detection. The BER is estimated as a function of E_b/N_0 and the simulated performance is compared with the theoretical BER expression for QPSK in AWGN. In addition, supporting experiments commonly used to assess practical receiver behavior are provided: constellation diagrams at multiple SNR values, a PSD comparison between rectangular and root raised cosine (RRC) pulse shaping, a training-length sweep illustrating the trade-off between estimation quality and overhead, and eye diagrams for AWGN and a simple two-tap multipath channel. Overall, the BER curve closely follows theory under ideal conditions, and the estimated synchronization or phase case remains close to ideal for the considered training length and AWGN-only channel model.

1 Background and Problem Formulation

QPSK is a fundamental modulation scheme and a standard reference point for evaluating receiver processing such as matched filtering, timing recovery, and carrier phase estimation. The objective of this project is to implement and validate a QPSK baseband simulation chain in MATLAB and to quantify its BER performance over an AWGN channel. The project tasks are: (i) implement the missing functions in the provided skeleton (mapping, synchronization, phase estimation, detection), (ii) simulate BER versus E_b/N_0 and compare to the theoretical BER curve, and (iii) analyze practical aspects of the link through additional plots and parameter sweeps (constellation, PSD, training length, and eye diagrams).

2 System Model

2.1 Transmitter

Binary data bits $b[n] \in \{0, 1\}$ are Gray-mapped into complex QPSK symbols. With two bits per symbol, one convenient representation is

$$d_k = \frac{1}{\sqrt{2}} ((1 - 2b_{I,k}) + j(1 - 2b_{Q,k})), \quad b_{I,k}, b_{Q,k} \in \{0, 1\}.$$

The symbol sequence is upsampled by a factor Q and pulse-shaped by a discrete-time pulse $p[n]$ to form the transmit waveform $x[n]$.

2.2 Channel

The baseband channel is modeled as complex AWGN:

$$r[n] = x[n] + w[n],$$

where $w[n]$ is circularly symmetric complex Gaussian noise. The noise variance is chosen as a function of E_b/N_0 and the pulse energy such that the desired SNR is achieved.

2.3 Receiver

The receiver applies a matched filter $p^*[-n]$ and performs: (i) timing synchronization based on correlation with a known training sequence, (ii) downsampling by Q to obtain symbol-rate samples, (iii) phase estimation based on training symbols followed by phase correction, and (iv) hard-decision detection and bit demapping.

3 Theoretical BER for QPSK in AWGN

For Gray-coded QPSK in AWGN, the bit error probability equals that of BPSK:

$$P_b = Q\left(\sqrt{2\frac{E_b}{N_0}}\right) = \frac{1}{2} \operatorname{erfc}\left(\sqrt{\frac{E_b}{N_0}}\right). \quad (1)$$

For Gray-coded QPSK in AWGN, the in-phase and quadrature components can be viewed as two independent BPSK channels, each carrying one bit with energy E_b . Therefore, the bit error probability equals the BPSK bit error probability:

$$P_b = Q\left(\sqrt{2E_b/N_0}\right).$$

Using the identity $Q(x) = \frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$ yields $P_b = \frac{1}{2}\operatorname{erfc}\left(\sqrt{E_b/N_0}\right)$, which is used for the theory curve.

This expression is used as the reference curve to validate the simulation chain.

4 Simulation Setup

The simulation is implemented in MATLAB using the provided skeleton and the required functions. Key parameters are summarized in Table 1.

Table 1: Main simulation parameters used in the experiments.

Parameter	Value
Modulation	Gray-coded QPSK
Bits per symbol k	2
Samples per symbol Q	8
Training length	100 bits (default), sweep in Fig. 4
Data bits per block	1000 bits
Guard bits	10 bits
Channel	AWGN (plus optional two-tap multipath for eye diagram)
Pulse shaping	Rectangular; PSD compared to RRC in Fig. 3

Two BER scenarios are evaluated: First one is "Perfect synchronization + perfect phase case". It is a reference timing instant is used and no phase error is assumed. Second one is "Estimated synchronization + estimated phase case". Timing and phase are estimated using the training sequence. In addition to BER, the following supporting results are generated: constellation diagrams (Fig. 2), PSD comparison between rectangular and RRC pulse shaping (Fig. 3), training length sweep and overhead (Fig. 4), and eye diagrams for AWGN and multipath (Fig. 5).

5 Results

5.1 BER versus E_b/N_0

Figure 1 compares simulated BER with the theoretical curve in (1). The ideal (perfect) case closely follows theory, confirming consistency of the mapping, noise scaling, and receiver processing. The estimated synchronization/phase case remains close to ideal under AWGN with the chosen training length.

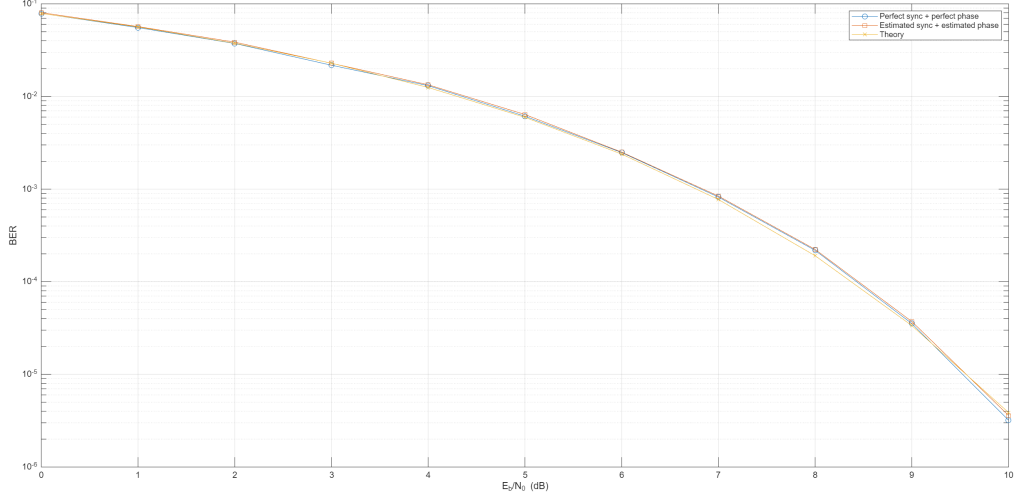
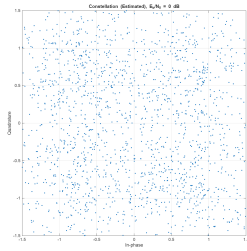


Figure 1: BER versus E_b/N_0 for perfect sync/phase, estimated sync/phase, and theory.

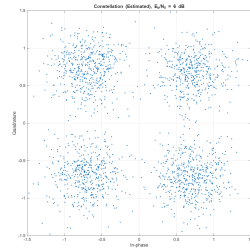
The agreement with the theoretical BER is expected for the perfect synchronization/phase case in an AWGN-only channel. At high E_b/N_0 , deviations or missing points may occur when only a limited number of errors is observed. In such cases, the reported BER should be interpreted as an upper bound determined by the simulated number of bits. Performance under estimated synchronization/phase can be improved by increasing the training length, widening/adapting the synchronization search window, and using a deterministic training sequence with better autocorrelation properties.

5.2 Constellation diagrams

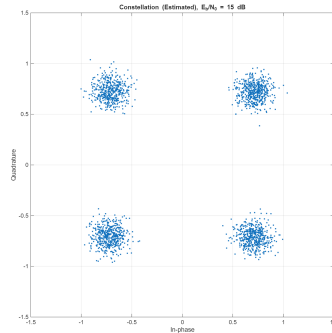
Figure 2 shows the constellation after timing synchronization and phase correction for three SNR values. As E_b/N_0 increases, the received clusters become tighter and more concentrated around the ideal QPSK points.



(a) $E_b/N_0 = 0$ dB



(b) $E_b/N_0 = 6$ dB



(c) $E_b/N_0 = 15$ dB

Figure 2: Constellation diagrams (estimated sync and phase correction).

5.3 PSD comparison: rectangular vs RRC pulse shaping

Figure 3 compares the power spectral density for rectangular and RRC pulse shaping. As expected, RRC significantly reduces out-of-band sidelobes and yields better spectral confinement.

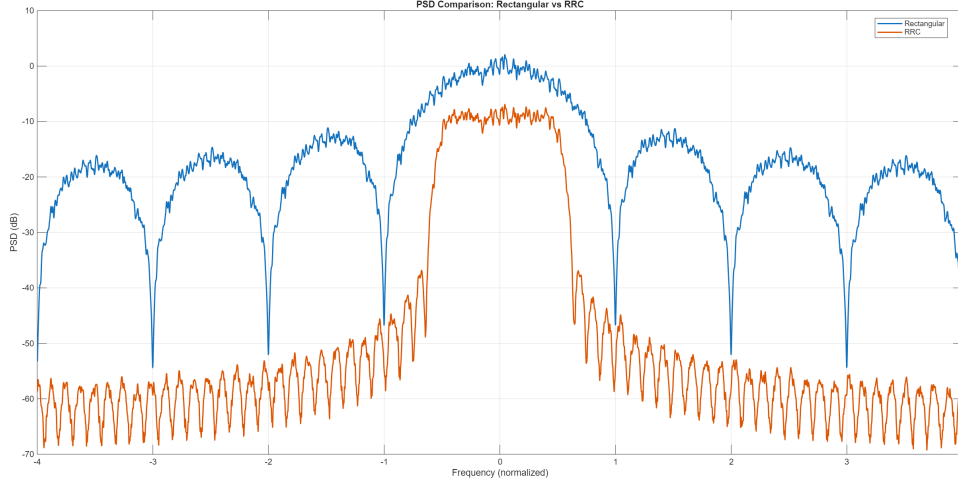
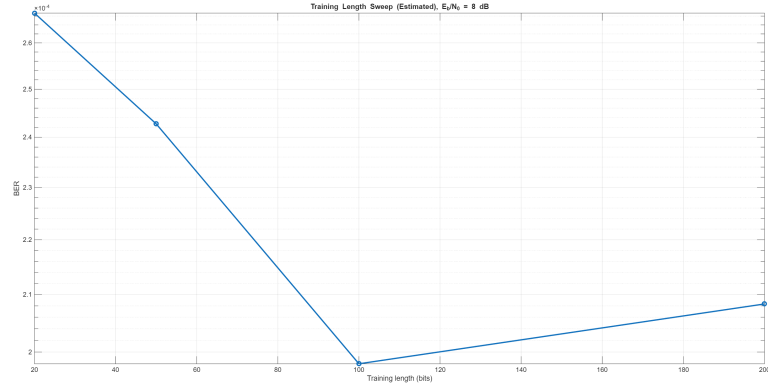


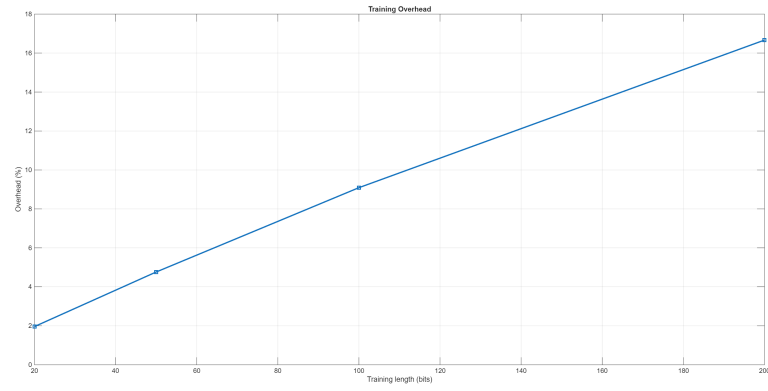
Figure 3: PSD comparison between rectangular and RRC pulse shaping.

5.4 Training length sweep and overhead

Figure 4 illustrates how the training length affects BER (estimated case) at $E_b/N_0 = 8$ dB, and the associated overhead. Short training sequences provide less reliable timing/phase estimation, while longer training increases overhead.



(a) BER versus training length

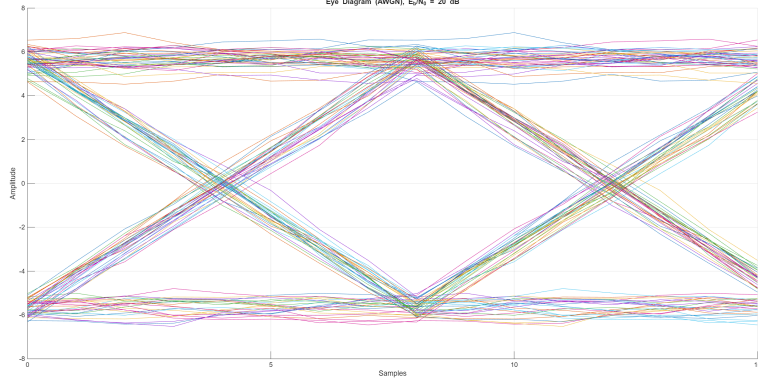


(b) Overhead versus training length

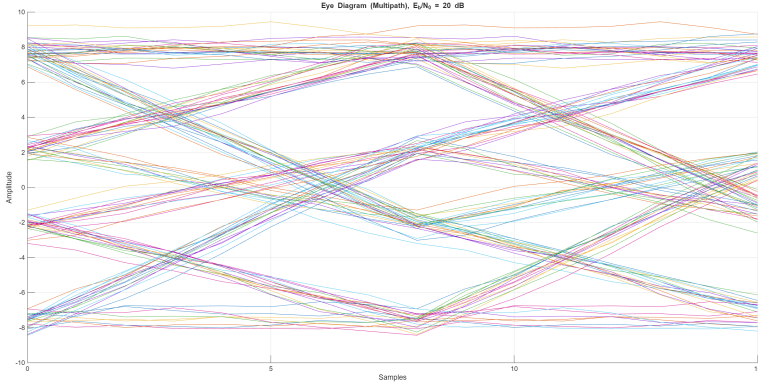
Figure 4: Training-length sweep (estimated) and corresponding overhead.

5.5 Eye diagrams: AWGN and multipath

Figure 5 shows eye diagrams based on the matched filter output for AWGN and for a simple two-tap multipath channel. Compared to AWGN, multipath introduces intersymbol interference (ISI), which visually reduces the eye opening and increases timing ambiguity.



(a) AWGN, $E_b/N_0 = 20$ dB



(b) Multipath, $E_b/N_0 = 20$ dB

Figure 5: Eye diagrams for AWGN and a two-tap multipath channel.

6 Discussion

The BER results validate the implemented QPSK baseband chain: under ideal synchronization and phase, the simulated BER closely follows the theoretical expression in (1). When synchronization and phase are estimated from training symbols, the BER remains close to the ideal case in an AWGN-only channel. It indicates that the correlation based timing estimator and training based phase estimator are effective for the selected training length and search window. The constellation diagrams provide a consistent qualitative interpretation of the BER curve: increased E_b/N_0 leads to reduced noise spread around the ideal QPSK points. The PSD comparison confirms the expected spectral benefit of RRC pulse shaping compared to rectangular pulses, which exhibit prominent sidelobes and poor spectral confinement. Finally, the eye diagrams highlight that while AWGN primarily adds vertical noise, multipath introduces ISI that closes the eye and degrades timing robustness, explaining why synchronization becomes more challenging under dispersive channels.

7 Conclusions

A complete Gray-coded QPSK baseband simulation chain was implemented and evaluated in MATLAB. The simulated BER under perfect synchronization and phase matches the theoretical QPSK/BPSK BER curve in AWGN. Training-based timing and phase estimation achieved near-ideal performance in AWGN for the chosen training length. Additional experiments (constellation, PSD, training sweep, and eye diagrams) provide complementary validation and illustrate practical trade-offs, especially the spectral advantages of RRC shaping and the ISI impact of multipath channels.

References

[1] EQ2310 course notes and project assignment handout.

Code Enclosure

All MATLAB source files used to generate the results are included below.

listings xcolor

simulation.m

```
1 % Skeleton code for simulation chain
2 % History:
3 %   2000-06-28   written /Stefan Parkvall
4 %   2001-10-22   modified /George Jongren
5 clear;
6 clc;
7 close all;
8 % Initialization
9 EbN0_db = 0:10; % Eb/N0 values to simulate (in dB)
10 nr_bits_per_symbol = 2; % Corresponds to k in the report
11 nr_guard_bits = 10; % Size of guard sequence (in nr bits)
12 % Guard bits are appended to transmitted bits so
13 % that the transients in the beginning and end
14 % of received sequence do not affect the samples
15 % which contain the training and data symbols.
16 nr_data_bits = 1000; % Size of each data sequence (in nr bits)
17 nr_training_bits = 100; % Size of training sequence (in nr bits)
18 nr_blocks = 50; % The number of blocks to simulate (minimum)
19 Q = 8; % Number of samples per symbol in baseband
20 % Extra simulation controls
21 targetErrors = 200; % target errors per Eb/N0 point (per curve)
22 maxTotalBits = 5e6; % safety cap (per Eb/N0 point)
23 nr_blocks_max = 50000; % max blocks cap (per Eb/N0 point)
24 % Define the pulse-shape used in the transmitter.
25 % Pick one of the pulse shapes below or experiemnt
26 % with a pulse of your own.
27 pulse_shape = ones(1, Q);
28 %pulse_shape = root_raised_cosine(Q);
29 % Matched filter impulse response.
30 mf_pulse_shape = fliplr(pulse_shape);
31 % Synchronization search window (initial suggestion)
32 t_start=1+Q*nr_guard_bits/2;
33 t_end=t_start+50;
34 % Fixed training sequence (keeps things controlled)
35 b_train_fixed = training_sequence(nr_training_bits);
36 % Compute a reference sampling time for "perfect synchronization" (noise-free)
37 b_data0 = random_data(nr_data_bits);
38 b_guard0 = random_data(nr_guard_bits);
39 b0 = [b_guard0 b_train_fixed b_data0 b_guard0];
40 d0 = qpsk(b0);
41 tx0 = upfirdn(d0, pulse_shape, Q, 1);
42 rx0 = tx0;
43 mf0 = conv(mf_pulse_shape, rx0);
44 t_start0 = max(1, round(t_start));
45 t_end0 = min(length(mf0), round(t_end));
46 t_samp_ref = sync(mf0, b_train_fixed, Q, t_start0, t_end0);
47 % Loop over different values of Eb/No.
48 nr_errors_perf = zeros(1, length(EbN0_db)); % Error counter (perfect)
49 nr_errors_est = zeros(1, length(EbN0_db)); % Error counter (estimated)
50 nr_bits_count = zeros(1, length(EbN0_db)); % Total data bits simulated
51 for snr_point = 1:length(EbN0_db)
52     err_perf = 0;
53     err_est = 0;
54     bits_used = 0;
55     % Loop over blocks to get sufficient statistics.
56     for blk = 1:nr_blocks_max
57         % Stop if we have enough errors for both curves AND we have simulated at least nr_blocks
58         if (blk >= nr_blocks) && (err_perf >= targetErrors) && (err_est >= targetErrors)
59             break;
60     end
```



```

61 % Stop if we hit the bit cap
62 if bits_used >= maxTotalBits
63     break;
64 end
65 %%%
66 %%% Transmitter
67 %%%
68 % Generate training sequence.
69 b_train = b_train_fixed;
70 % Generate random source data {0, 1}.
71 b_data = random_data(nr_data_bits);
72 % Generate guard sequence.
73 b_guard = random_data(nr_guard_bits);
74 % Multiplex training and data into one sequence.
75 b = [b_guard b_train b_data b_guard];
76 % Map bits into complex-valued QPSK symbols.
77 d = qpsk(b);
78 % Upsample the signal, apply pulse shaping.
79 tx = upfirdn(d, pulse_shape, Q, 1);
80 %%%
81 %%% AWGN Channel
82 %%%
83 % Compute variance of complex noise according to report.
84 sigma_sqr = norm(pulse_shape)^2 / nr_bits_per_symbol / 10^(EbN0_db(snr_point)/10);
85 % Create noise vector.
86 n = sqrt(sigma_sqr/2)*(randn(size(tx))+1j*randn(size(tx)));
87 % Received signal.
88 rx = tx + n;
89 %%%
90 %%% Receiver
91 %%%
92 % Matched filtering.
93 mf=conv(mf_pulse_shape,rx);
94 % Make sure sync search window is inside mf
95 t_start1 = max(1, round(t_start));
96 t_end1 = min(length(mf), round(t_end));
97 % ---- Estimated synchronization ----
98 t_samp_est = sync(mf, b_train, Q, t_start1, t_end1);
99 r_end_est = t_samp_est + Q*(nr_training_bits+nr_data_bits)/2 - 1;
100 if r_end_est > length(mf) || t_samp_est < 1
101     continue;
102 end
103 % Down sampling. Only training+data samples are kept.
104 r_est = mf(t_samp_est:Q:r_end_est);
105 % Phase estimation and correction.
106 phihat_est = phase_estimation(r_est, b_train);
107 r_est = r_est * exp(-1j*phihat_est);
108 % Make decisions.
109 bhat_est = detect(r_est);
110 % Count errors on data bits only.
111 temp_est = bhat_est(1+nr_training_bits:nr_training_bits+nr_data_bits) ~= b_data;
112 err_est = err_est + sum(temp_est);
113 % ---- Perfect synchronization + perfect phase ----
114 t_samp_perf = t_samp_ref;
115 r_end_perf = t_samp_perf + Q*(nr_training_bits+nr_data_bits)/2 - 1;
116 if r_end_perf > length(mf) || t_samp_perf < 1
117     continue;
118 end
119 r_perf = mf(t_samp_perf:Q:r_end_perf);
120 phihat_perf = 0;
121 r_perf = r_perf * exp(-1j*phihat_perf);
122 bhat_perf = detect(r_perf);
123 temp_perf = bhat_perf(1+nr_training_bits:nr_training_bits+nr_data_bits) ~= b_data;
124 err_perf = err_perf + sum(temp_perf);
125 bits_used = bits_used + nr_data_bits;
126 % Next block.
127 end
128 nr_errors_perf(snr_point) = err_perf;
129 nr_errors_est(snr_point) = err_est;
130 nr_bits_count(snr_point) = bits_used;
131 % Next Eb/No value.
132 end
133 % Compute the BER.
134 BER_perf = nr_errors_perf ./ nr_bits_count;
135 BER_est = nr_errors_est ./ nr_bits_count;

```

```

136 % For semilogy: replace zeros with an upper bound 0.5/Nbits (so points still appear)
137 BER_perf_plot = BER_perf;
138 BER_est_plot = BER_est;
139 idxOp = (nr_errors_perf == 0) & (nr_bits_count > 0);
140 idxOe = (nr_errors_est == 0) & (nr_bits_count > 0);
141 BER_perf_plot(idxOp) = 0.5 ./ nr_bits_count(idxOp);
142 BER_est_plot(idxOe) = 0.5 ./ nr_bits_count(idxOe);
143 disp(BER_perf)
144 disp(BER_est)
145 EbN0 = 10.^(EbN0_db/10);
146 BER_theory = 0.5*erfc(sqrt(EbN0));
147 figure
148 semilogy(EbN0_db, BER_perf_plot, 'o-')
149 grid on
150 hold on
151 semilogy(EbN0_db, BER_est_plot, 's-')
152 semilogy(EbN0_db, BER_theory, 'x-')
153 xlabel('E_b/N_0 (dB)')
154 ylabel('BER')
155 legend('Perfect sync + perfect phase','Estimated sync + estimated phase','Theory')

```

project_extras.m

```

1 clear;
2 clc;
3 close all;
4 EbN0_list_const = [0 6 15];
5 EbN0_train_sweep = 8;
6 EbN0_eye = 20;
7 k = 2;
8 nr_guard_bits = 10;
9 nr_data_bits = 1000;
10 Q = 8;
11 beta = 0.22;
12 spanSymbols = 4;
13 pulse_rect = ones(1, Q);
14 pulse_rrc = rrc_pulse(Q, beta, spanSymbols);
15 t_start = round(1 + Q*nr_guard_bits/2);
16 t_end = round(t_start + 50);
17 targetErrors = 200;
18 maxTotalBits = 2e6;
19 maxBlocks = 5000;
20 tau_mp = Q;
21 rng(1);
22 disp('1) Generating constellation diagrams...');
23 make_constellation_figs(EbN0_list_const, pulse_rect, Q, k, nr_guard_bits, nr_data_bits,
    t_start, t_end, maxBlocks);
24 disp('2) Generating PSD comparison (Rectangular vs RRC)...');
25 make_psd_fig(pulse_rect, pulse_rrc, Q);
26 disp('3) Running training length sweep...');
27 make_training_sweep_fig(EbN0_train_sweep, pulse_rect, Q, k, nr_guard_bits, nr_data_bits,
    t_start, t_end, targetErrors, maxTotalBits, maxBlocks);
28 disp('4) Generating eye diagrams (AWGN and Multipath)...');
29 make_eye_figs(EbN0_eye, pulse_rect, Q, k, nr_guard_bits, nr_data_bits, t_start, t_end,
    tau_mp);
30 disp('All figures were saved.');
```

```

31 function make_constellation_figs(EbN0_list, pulse_shape, Q, k, nr_guard_bits, nr_data_bits,
    t_start, t_end, maxBlocks)
32 mf_pulse_shape = fliplr(pulse_shape);
33 nr_training_bits = 100;
34 b_train = training_sequence(nr_training_bits);
35 d_train = qpsk(b_train);
36 NtrainSym = length(d_train);
37 for i = 1:numel(EbN0_list)
38     EbN0_db = EbN0_list(i);
39     sigma_sqr = norm(pulse_shape)^2 / k / (10^(EbN0_db/10));
40     r_collect = [];
41     blk = 0;
42     while (numel(r_collect) < 2000) && (blk < maxBlocks)
43         blk = blk + 1;
44         b_data = random_data(nr_data_bits);
45         b_guard = random_data(nr_guard_bits);
46         b = [b_guard b_train b_data b_guard];

```

```

47     d = qpsk(b);
48     tx = upfirdn(d, pulse_shape, Q, 1);
49     n = sqrt(sigma_sqr/2) * (randn(size(tx)) + 1j*randn(size(tx)));
50     rx = tx + n;
51     mf = conv(mf_pulse_shape, rx);
52     t_samp = sync(mf, b_train, Q, t_start, min(length(mf), t_end));
53     if isempty(t_samp) || t_samp < 1
54         continue;
55     end
56     r_end = t_samp + Q*(nr_training_bits + nr_data_bits)/2 - 1;
57     if r_end > length(mf)
58         continue;
59     end
60     r = mf(t_samp:Q:r_end);
61     phihat = phase_estimation(r, b_train);
62     r = r * exp(-1j*phihat);
63     if length(r) < NtrainSym
64         continue;
65     end
66     r_train = r(1:NtrainSym);
67     g = sum(conj(d_train).*r_train) / sum(abs(d_train).^2);
68     if g ~= 0
69         r = r / g;
70     end
71     idx0 = NtrainSym + 1;
72     if idx0 <= length(r)
73         r_collect = [r_collect r(idx0:end)];
74     end
75 end
76 figure;
77 set(gcf, 'Color', 'w');
78 plot(real(r_collect), imag(r_collect), '.');
79 axis equal;
80 grid on;
81 xlabel('In-phase');
82 ylabel('Quadrature');
83 title(['Constellation (Estimated), E_b/N_0 = ' num2str(EbN0_db) ' dB']);
84 xlim([-1.5 1.5]);
85 ylim([-1.5 1.5]);
86 ax = gca;
87 if isprop(ax, 'Toolbar')
88     ax.Toolbar.Visible = 'off';
89 end
90 saveas(gcf, ['const_' num2str(EbN0_db) 'dB.png']);
91 end
92 end
93 function make_psd_fig(pulse_rect, pulse_rrc, Q)
94     nr_training_bits = 100;
95     nr_data_bits = 4000;
96     nr_guard_bits = 10;
97     b_train = training_sequence(nr_training_bits);
98     b_data = random_data(nr_data_bits);
99     b_guard = random_data(nr_guard_bits);
100     b = [b_guard b_train b_data b_guard];
101     d = qpsk(b);
102     tx_rect = upfirdn(d, pulse_rect, Q, 1);
103     tx_rrc = upfirdn(d, pulse_rrc, Q, 1);
104     Fs = Q;
105     nfft = 4096;
106     [f1, P1] = compute_psd(tx_rect, Fs, nfft);
107     [f2, P2] = compute_psd(tx_rrc, Fs, nfft);
108     figure;
109     set(gcf, 'Color', 'w');
110     plot(f1, 10*log10(P1), 'LineWidth', 1.5);
111     hold on;
112     plot(f2, 10*log10(P2), 'LineWidth', 1.5);
113     grid on;
114     xlabel('Frequency (normalized)');
115     ylabel('PSD (dB)');
116     legend('Rectangular', 'RRC', 'Location', 'best');
117     title('PSD Comparison: Rectangular vs RRC');
118     ax = gca;
119     if isprop(ax, 'Toolbar')
120         ax.Toolbar.Visible = 'off';
121     end

```

```

122 saveas(gcf, 'psd_rect_vs_rrc.png');
123 end
124 function make_training_sweep_fig(EbN0_db, pulse_shape, Q, k, nr_guard_bits, nr_data_bits,
    t_start, t_end, targetErrors, maxTotalBits, maxBlocks)
125 mf_pulse_shape = flipplr(pulse_shape);
126 train_list = [20 50 100 200];
127 BER_train = nan(size(train_list));
128 overhead = nan(size(train_list));
129 for i = 1:numel(train_list)
130     nr_training_bits = train_list(i);
131     b_train = training_sequence(nr_training_bits);
132     BER_train(i) = ber_point_estimated(EbN0_db, pulse_shape, mf_pulse_shape, Q, k,
        nr_guard_bits, nr_training_bits, nr_data_bits, b_train, t_start, t_end, targetErrors,
        maxTotalBits, maxBlocks);
133     overhead(i) = nr_training_bits / (nr_training_bits + nr_data_bits);
134     fprintf('Training bits: %d -> BER: %.6g\n', nr_training_bits, BER_train(i));
135 end
136 figure;
137 set(gcf, 'Color', 'w');
138 semilogy(train_list, BER_train, 'o-', 'LineWidth', 2);
139 grid on;
140 xlabel('Training length (bits)');
141 ylabel('BER');
142 title(['Training Length Sweep (Estimated), Eb/N0 = ' num2str(EbN0_db) ' dB']);
143 ax = gca;
144 if isprop(ax, 'Toolbar')
145     ax.Toolbar.Visible = 'off';
146 end
147 saveas(gcf, 'training_sweep_ber.png');
148 figure;
149 set(gcf, 'Color', 'w');
150 plot(train_list, overhead*100, 's-', 'LineWidth', 2);
151 grid on;
152 xlabel('Training length (bits)');
153 ylabel('Overhead (%)');
154 title('Training Overhead');
155 ax = gca;
156 if isprop(ax, 'Toolbar')
157     ax.Toolbar.Visible = 'off';
158 end
159 saveas(gcf, 'training_sweep_overhead.png');
160 end
161 function BER = ber_point_estimated(EbN0_db, pulse_shape, mf_pulse_shape, Q, k, nr_guard_bits,
    nr_training_bits, nr_data_bits, b_train, t_start, t_end, targetErrors, maxTotalBits,
    maxBlocks)
162 sigma_sqr = norm(pulse_shape)^2 / k / (10^(EbN0_db/10));
163 err = 0;
164 bits_used = 0;
165 blk = 0;
166 while (err < targetErrors) && (bits_used < maxTotalBits) && (blk < maxBlocks)
167     blk = blk + 1;
168     b_data = random_data(nr_data_bits);
169     b_guard = random_data(nr_guard_bits);
170     b = [b_guard b_train b_data b_guard];
171     d = qpsk(b);
172     tx = upfirdn(d, pulse_shape, Q, 1);
173     n = sqrt(sigma_sqr/2) * (randn(size(tx)) + 1j*randn(size(tx)));
174     rx = tx + n;
175     mf = conv(mf_pulse_shape, rx);
176     t_samp = sync(mf, b_train, Q, t_start, min(length(mf), t_end));
177     if isempty(t_samp) || t_samp < 1
178         continue;
179     end
180     r_end = t_samp + Q*(nr_training_bits + nr_data_bits)/2 - 1;
181     if r_end > length(mf)
182         continue;
183     end
184     r = mf(t_samp:Q:r_end);
185     phihat = phase_estimation(r, b_train);
186     r = r * exp(-1j*phihat);
187     bhat = detect(r);
188     temp = bhat(1+nr_training_bits:nr_training_bits+nr_data_bits) ~= b_data;
189     err = err + sum(temp);
190     bits_used = bits_used + nr_data_bits;
191 end

```

```

192 if bits_used == 0
193     BER = NaN;
194 else
195     if err == 0
196         BER = 0.5 / bits_used;
197     else
198         BER = err / bits_used;
199     end
200 end
201 end
202 function make_eye_figs(EbN0_db, pulse_shape, Q, k, nr_guard_bits, nr_data_bits, t_start,
    t_end, tau)
203 mf_pulse_shape = fliplr(pulse_shape);
204 nr_training_bits = 100;
205 b_train = training_sequence(nr_training_bits);
206 sigma_sqr = norm(pulse_shape)^2 / k / (10^(EbN0_db/10));
207 b_data = random_data(nr_data_bits);
208 b_guard = random_data(nr_guard_bits);
209 b = [b_guard b_train b_data b_guard];
210 d = qpsk(b);
211 tx = upfirdn(d, pulse_shape, Q, 1);
212 n_awgn = sqrt(sigma_sqr/2) * (randn(size(tx)) + 1j*randn(size(tx)));
213 rx_awgn = tx + n_awgn;
214 mf_awgn = conv(mf_pulse_shape, rx_awgn);
215 t_samp_awgn = sync(mf_awgn, b_train, Q, t_start, min(length(mf_awgn), t_end));
216 if isempty(t_samp_awgn) || t_samp_awgn < 1
217     t_samp_awgn = 1;
218 end
219 x_eye_awgn = real(mf_awgn(t_samp_awgn:end));
220 figure;
221 set(gcf, 'Color', 'w');
222 plot_eye(x_eye_awgn, Q, 2, 120, Q);
223 grid on;
224 xlabel('Samples');
225 ylabel('Amplitude');
226 title(['Eye Diagram (AWGN), E_b/N_0 = ' num2str(EbN0_db) ' dB']);
227 ax = gca;
228 if isprop(ax, 'Toolbar')
229     ax.Toolbar.Visible = 'off';
230 end
231 saveas(gcf, 'eye_awgn.png');
232 rx_mp = two_tap_multipath(tx, tau);
233 n_mp = sqrt(sigma_sqr/2) * (randn(size(rx_mp)) + 1j*randn(size(rx_mp)));
234 rx_mp = rx_mp + n_mp;
235 mf_mp = conv(mf_pulse_shape, rx_mp);
236 t_samp_mp = sync(mf_mp, b_train, Q, t_start, min(length(mf_mp), t_end));
237 if isempty(t_samp_mp) || t_samp_mp < 1
238     t_samp_mp = 1;
239 end
240 x_eye_mp = real(mf_mp(t_samp_mp:end));
241 figure;
242 set(gcf, 'Color', 'w');
243 plot_eye(x_eye_mp, Q, 2, 120, Q);
244 grid on;
245 xlabel('Samples');
246 ylabel('Amplitude');
247 title(['Eye Diagram (Multipath), E_b/N_0 = ' num2str(EbN0_db) ' dB']);
248 ax = gca;
249 if isprop(ax, 'Toolbar')
250     ax.Toolbar.Visible = 'off';
251 end
252 saveas(gcf, 'eye_multipath.png');
253 end
254 function plot_eye(x, Q, spanSymbols, nTraces, stride)
255 x = x(:).';
256 L = spanSymbols * Q;
257 maxStart = length(x) - L + 1;
258 if maxStart < 1
259     return;
260 end
261 starts = 1:stride:maxStart;
262 if numel(starts) > nTraces
263     starts = starts(1:nTraces);
264 end
265 hold on;

```

```

266 for k = 1:numel(starts)
267     seg = x(starts(k):starts(k)+L-1);
268     plot(0:L-1, seg);
269 end
270 hold off;
271 end
272 function [f, Pxx] = compute_psd(x, Fs, nfft)
273 x = x(:);
274 if exist('pwelch','file') == 2
275     [Pxx, f] = pwelch(x, hamming(min(length(x),1024)), [], nfft, Fs, 'centered');
276 else
277     N = min(length(x), nfft);
278     w = hamming(N);
279     xw = x(1:N) .* w;
280     X = fftshift(fft(xw, nfft));
281     Pxx = (abs(X).^2) / (Fs*sum(w.^2));
282     f = Fs*((-nfft/2):(nfft/2-1))/nfft;
283 end
284 end
285 function h = rrc_pulse(Q, beta, spanSymbols)
286 T = 1;
287 t = (-spanSymbols*Q:spanSymbols*Q) / Q;
288 h = zeros(size(t));
289 for i = 1:numel(t)
290     ti = t(i);
291     if abs(ti) < 1e-12
292         h(i) = (1 + beta*(4/pi - 1)) / sqrt(T);
293     elseif abs(abs(ti) - T/(4*beta)) < 1e-12
294         h(i) = (beta/sqrt(2*T)) * ((1 + 2/pi)*sin(pi/(4*beta)) + (1 - 2/pi)*cos(pi/(4*beta)));
295     else
296         num = cos((1+beta)*pi*ti/T) + (T/(4*beta*ti))*sin((1-beta)*pi*ti/T);
297         den = 1 - (4*beta*ti/T)^2;
298         h(i) = (4*beta/(pi*sqrt(T))) * (num/den);
299     end
300 end
301 h = h / norm(h);
302 end
303 function y = two_tap_multipath(x, tau)
304 a1 = sqrt(3/4);
305 a2 = sqrt(1/4);
306 x = x(:).';
307 if tau < 1
308     y = (a1 + a2) * x;
309     y = y(:);
310     return;
311 end
312 if tau >= length(x)
313     y = a1*x;
314     y = y(:);
315     return;
316 end
317 y = a1*x + a2*[zeros(1,tau) x(1:end-tau)];
318 y = y(:);
319 end

```

qpsk.m

```

1 function d = qpsk(b)
2 b = b(:).';
3 if mod(length(b),2) ~= 0
4     error('Input bit vector length must be even.');
```

```

5 end
6 bI = double(b(1:2:end));
7 bQ = double(b(2:2:end));
8 I = 1 - 2*bI;
9 Q = 1 - 2*bQ;
10 d = (I + 1j*Q)/sqrt(2);
11 end

```

detect.m

```

1 function bhat = detect(r)
2 r = r(:).';
3 bI = real(r) < 0;
4 bQ = imag(r) < 0;
5 bhat = zeros(1, 2*length(r));
6 bhat(1:2:end) = double(bI);
7 bhat(2:2:end) = double(bQ);
8 end

```

sync.m

```

1 function t_samp = sync(mf, b_train, Q, t_start, t_end)
2 mf = mf(:).';
3 d_train = qpsk(b_train);
4 N = length(d_train);
5 best_metric = -Inf;
6 best_t = t_start;
7 t_start = max(1, round(t_start));
8 t_end = min(length(mf), round(t_end));
9 for t = t_start:t_end
10     last_idx = t + Q*(N-1);
11     if last_idx > length(mf)
12         break
13     end
14     r_try = mf(t:Q:last_idx);
15     c = sum(conj(d_train) .* r_try);
16     metric = abs(c);
17     if metric > best_metric
18         best_metric = metric;
19         best_t = t;
20     end
21 end
22 t_samp = best_t;
23 end

```

phase_estimation.m

```

1 function phihat = phase_estimation(r, b_train)
2 r = r(:).';
3 d_train = qpsk(b_train);
4 N = length(d_train);
5 if length(r) < N
6     error('Received sequence is shorter than training sequence.');
```

```

7 end
8 r_train = r(1:N);
9 z = sum(conj(d_train) .* r_train);
10 if z == 0
11     phihat = 0;
12 else
13     phihat = angle(z);
14 end
15 end

```

training_sequence.m

```

1 function b = training_sequence(nr_training_bits)
2 % b = training_sequence(nr_training_bits)
3 %
4 % Generate a training sequence consisting of n bits. Currently, a random
5 % sequence is used, but a deterministic sequence with better
6 % autocorrelation properties could be used instead.
7 %
8 % Input
9 %   nr_training_bits = length of training sequence (should be an even number)
10 %
11 % Output
12 %   b = training sequence {0/1}
13 %
14 % History:

```

```

15 % 2000-09-28 written /Stefan Parkvall
16 % 2001-10-21 modified / George J ngren
17 b = (rand(1, nr_training_bits) > .5);

```

random_data.m

```

1 function b = random_data(nr_data_bits)
2 % b = random_data(nr_data_bits)
3 %
4 % Generates a block of equiprobable random binary data, {0, 1}
5 %
6 % Input:
7 %   nr_data_bits = the number of bits to generate
8 %
9 % Output:
10 %   b = random data bits, {0, 1}
11 % History:
12 %   2000-06-26 written /Stefan Parkvall
13 %   2001-10-21 modified /George J ngren
14 b = (rand(1, nr_data_bits) > .5);

```

multipath.m

```

1 function y = multipath(x, tau, h)
2 % y = multipath(x, tau, h)
3 %
4 % A simple multipath channel with two paths. The amplification of respective
5 % path is sqrt(3/4) and sqrt(1/4) (unless h is given as an input
6 % argument). No noise is added, but the AWGN channel can be put afterwards
7 % to take care of this.
8 %
9 % Input:
10 %   x = the transmitted signal
11 %   tau = the delay between the first and second path measured in samples
12 %   h = vector of tap coefficients (optional)
13 %
14 % Output:
15 %   y = the sum of the two paths
16 % 2000-08-22 written /Stefan
17 if nargin < 3
18   h = [sqrt(3/4) ; sqrt(1/4)];
19 end
20 y = h(1)*x + h(2)*[zeros(1, tau) x(1:end-tau)];

```

root_raised_cosine.m

```

1 function pulse_shape = root_raised_cosine(Q, b, trunc)
2 % h = root_raised_cosine(Q, b, trunc)
3 %
4 % Computes a truncated root raised cosine FIR filter.
5 %
6 % Input:
7 %   Q = number of samples per symbol
8 %   b = roll-off factor (optional, default 0.22)
9 %   trunc = truncation, computes impulse response in the interval
10 %          [-trunc*Tsymb , trunc*Tsymb ] (optional, default trunc=5)
11 %
12 % Output:
13 %   pulse_shape = impulse response
14 %
15 % History:
16 %   2000-09-03 written /Stefan Parkvall
17 if nargin < 3
18   trunc = 5;
19 end
20 if nargin < 2
21   b = 0.22;
22 end
23 % The symbol time T is assumed to be unity.

```



```

24 T = 1*Q;
25 t = -trunc*Q:1:trunc*Q;
26 pulse_shape = 4 * b * ...
27     ( cos((1+b)*pi*t/T) + (1-b)*pi/(4*b) * sinc((1-b)*t/T) ) ./ ...
28     (pi*sqrt(T)*(1-16*b^2*t.^2/T^2));

```